

第 33 讲

Gradient Descent 梯度下降

参数到底怎样沿着负梯度一步一步降低 Loss

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】前面我们已经会用 `loss.backward()` 得到 Gradient，也会调用 `optimizer.step()` 更新参数。但如果只会写这两行代码，仍然不知道“更新为什么这样写、为什么是负号、Learning Rate 为什么会让训练变快或发散”。本讲暂时把 Optimizer 外壳拆掉，直接研究最基础的 Gradient Descent。

学习目标

学完本讲，你应该能够：

1. 解释 Gradient Descent 的输入、输出和内部更新步骤；
2. 从公式 $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$ 逐项解释每个符号；
3. 解释为什么使用负梯度方向，而不是正梯度方向；
4. 手算一个标量参数的一步和多步 Gradient Descent；
5. 把标量更新推广到向量、矩阵参数；
6. 理解 Learning Rate 只控制步长，不负责算梯度；
7. 区分 Backpropagation、Gradient Descent、Optimizer、SGD；
8. 看懂 PyTorch `torch.optim.SGD` 在最简单情况下做了什么；
9. 判断“训练 Loss 下降”能说明什么、不能说明什么；
10. 为下一讲的 SGD / Mini-batch SGD 建立严格基线。

1 先把四个概念彻底拆开

深度学习初学时最容易把下面四件事混在一起：

概念	真正负责的事情
Backpropagation	根据 Computational Graph 和 Chain Rule 计算 Gradient
Gradient	当前参数附近，Loss 对每个参数的局部敏感度
Gradient Descent	规定“拿到 Gradient 后，最基础地怎样更新参数”
Optimizer	对参数更新规则的工程封装；SGD、Momentum、Adam 都属于 Optimizer

一句话：Backward 负责“算方向信息”，Gradient Descent 负责“按照这个方向信息移动参数”，Optimizer 则把移动规则自动执行。

例如 PyTorch：

```
loss.backward()      # 计算梯度
optimizer.step()     # 使用优化器规则更新参数
```

两行代码在训练循环里紧挨着，但数学角色完全不同。

2 【原理】Gradient Descent 到底解决什么问题？

训练神经网络的核心任务可以写成：

$$\min_{\theta} L(\theta)$$

其中：

- θ ：所有可训练参数，可以是一维标量，也可以是上亿维参数向量；
- $L(\theta)$ ：当前参数下计算出来的 Loss；
- 目标：找到一组参数，让 Loss 尽可能小。

问题是：我们通常没有一个“直接把最优参数解出来”的简单公式。于是使用迭代思想：

从当前参数出发 → 看当前位置的坡度 → 往下降方向走一点 → 重新计算 → 重复

这就是 Gradient Descent 的基本思想。

3 【通俗解释】把 Loss Landscape 想成山地地图

第 21 讲已经学过 Loss Landscape。现在把它直接用于训练：

- 参数 θ ：你在地图上的位置；
- Loss：当前海拔；
- Gradient：最陡的上坡方向；
- Negative Gradient：局部最陡的下坡方向；

- Learning Rate：你每一步走多远。

因此最基础的策略是：

朝负梯度方向走一小步

“最陡下降方向”是当前位置附近的一阶局部结论，不是说从当前位置沿这条直线无限走下去永远都下降。因此步长过大仍可能越过谷底甚至发散。

4 【必要公式】 Gradient Descent 更新公式

最常见写法：

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

逐项拆开：

- t ：当前训练 Step；
- θ_t ：当前参数；
- $L(\theta_t)$ ：当前参数对应的 Loss；
- $\nabla L(\theta_t)$ ：当前点的 Gradient；
- η ：Learning Rate，学习率；
- $-\eta \nabla L$ ：这一次真正加到参数上的更新量；
- θ_{t+1} ：更新后的新参数。

5 为什么公式中有一个负号？

假设只有一个参数 w ，若：

$$\frac{dL}{dw} > 0$$

表示当前 w 稍微增大，Loss 会增大。因此想降低 Loss，应该让 w 减小：

$$w_{new} = w - \eta \frac{dL}{dw}$$

若：

$$\frac{dL}{dw} < 0$$

说明当前 w 增大反而会让 Loss 下降。此时：

$$-\eta \frac{dL}{dw} > 0$$

更新公式会自动让 w 增大。

负号不是死规定，而是在说：“Gradient 指向上升最快，所以我要朝它相反的方向走。”

6 从一阶近似再看：为什么负梯度是自然选择？

如果参数发生一个很小的变化：

$$\Delta\theta$$

那么当前点附近可以近似写成：

$$L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^T \Delta\theta$$

这句话不要求你做严格高数证明，只要把它读成：

新 Loss $\approx$ 当前 Loss + “Gradient 与这一步移动方向的匹配程度”。

如果我们选择：

$$\Delta\theta = -\eta \nabla L$$

代入：

$$\Delta L \approx \nabla L^T (-\eta \nabla L) = -\eta \|\nabla L\|^2$$

因为：

$$\eta > 0, \quad \|\nabla L\|^2 \geq 0$$

所以：

$$\Delta L \lesssim 0$$

这就是“沿负梯度小步走，局部上预计会降低 Loss”的更严格直觉来源。

这里最重要的两个限定词是：局部、小步。Gradient Descent 没有承诺一步跳到全局最优，它只利用当前点的一阶信息决定下一小步。

7 【具体数字例子】一步 Gradient Descent 完整手算

定义：

$$L(w) = (w - 3)^2$$

最低点显然在：

$$w = 3$$

假设现在：

$$w_0 = 8$$

则：

$$L(w_0) = (8 - 3)^2 = 25$$

导数：

$$\frac{dL}{dw} = 2(w - 3)$$

所以在 $w = 8$ 处：

$$\frac{dL}{dw} = 10$$

设 Learning Rate：

$$\eta = 0.1$$

更新：

$$w_1 = 8 - 0.1 \times 10 = 7$$

新的 Loss：

$$L(w_1) = (7 - 3)^2 = 16$$

因此：

$$25 \rightarrow 16$$

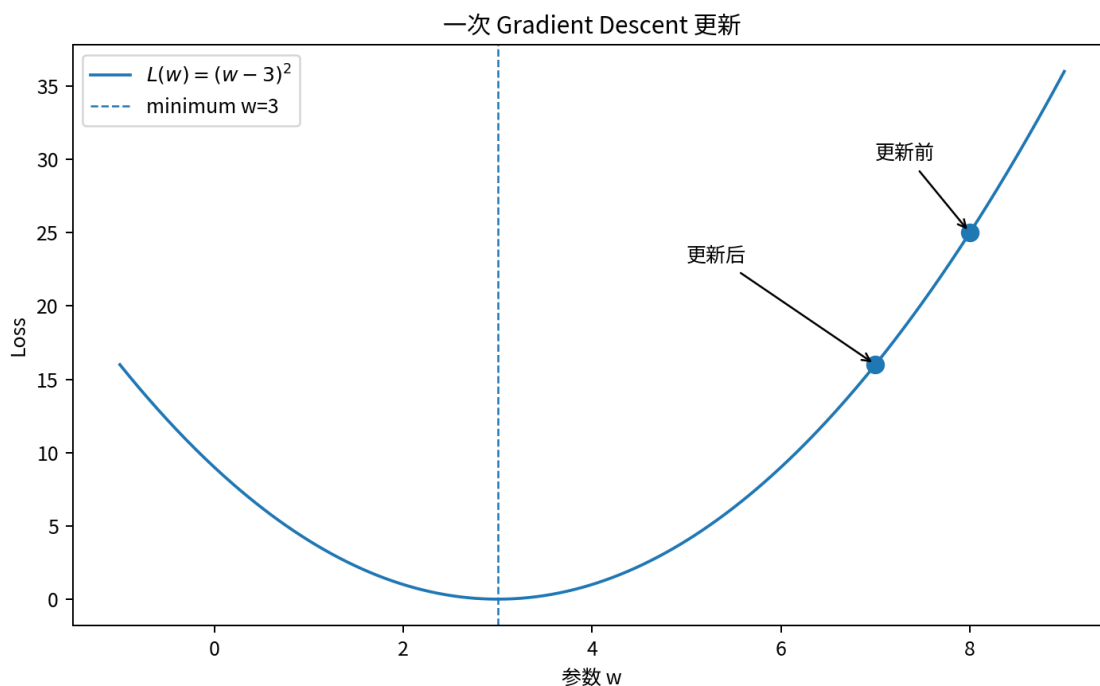


图 1: 从 $w = 8$ 沿负梯度方向做一次更新

这一小步中没有任何神秘过程：先算 $\text{Gradient}=10$ ，再乘学习率 0.1 得到步长 1 ，最后把参数从 8 改成 7 。

8 多步更新：为什么参数会逐渐靠近最低点？

继续使用：

$$L(w) = (w - 3)^2$$

$$w_{t+1} = w_t - 0.1 \cdot 2(w_t - 3)$$

前三步：

$$w_0 = 8$$

$$w_1 = 7$$

$$w_2 = 6.2$$

$$w_3 = 5.56$$

实验中到第 10 步：

$$w \approx 3.5369, \quad L \approx 0.2882$$

到第 25 步：

$$w \approx 3.0189, \quad L \approx 0.0003568$$

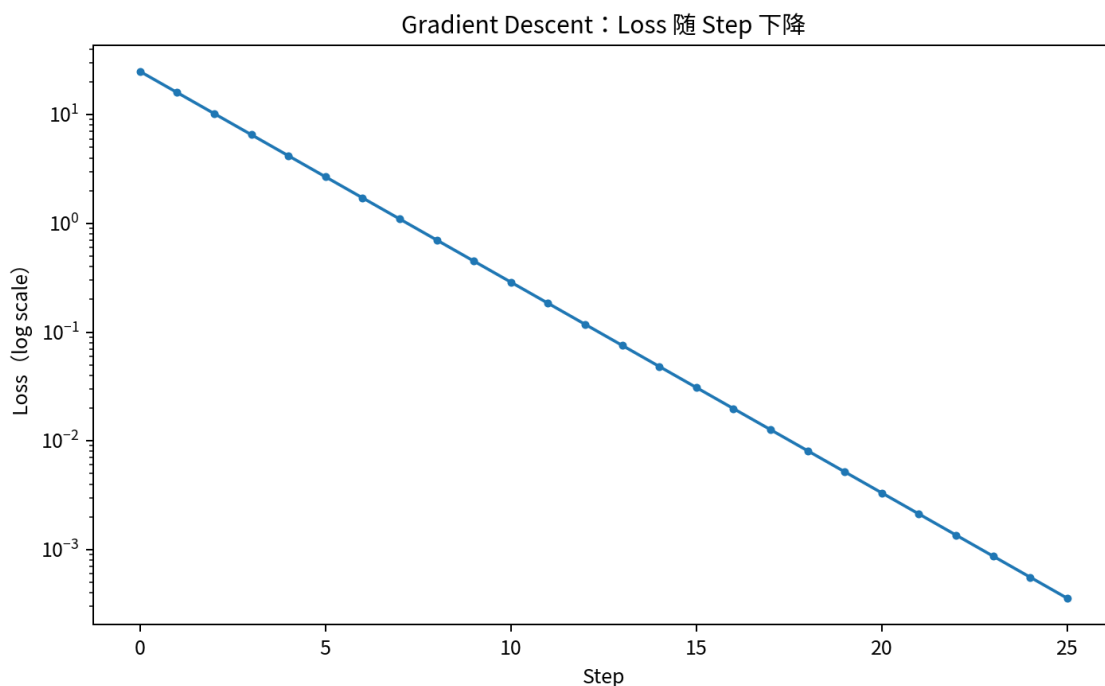


图 2: 多步 Gradient Descent 后 Loss 逐渐下降

当越来越接近 $w = 3$ 时：

$$\frac{dL}{dw} = 2(w - 3)$$

本身也越来越接近 0，因此更新量自然变小。

9 Learning Rate：方向正确，步长仍然可能出问题

Gradient 提供的是方向与局部尺度，而 Learning Rate：

$$\eta$$

控制每次到底走多远。

同一初始点 $w_0 = 8$ ，本讲固定其他所有条件，只改变 Learning Rate：

Learning Rate	20 步后的现象	直觉
0.05	仍在 3.6079，较慢	步子偏小
0.2	几乎到 3	稳定且较快
0.9	左右跨过最低点但逐渐缩小	大步振荡式收敛
1.1	参数跑到约 194.688，Loss 巨大	发散

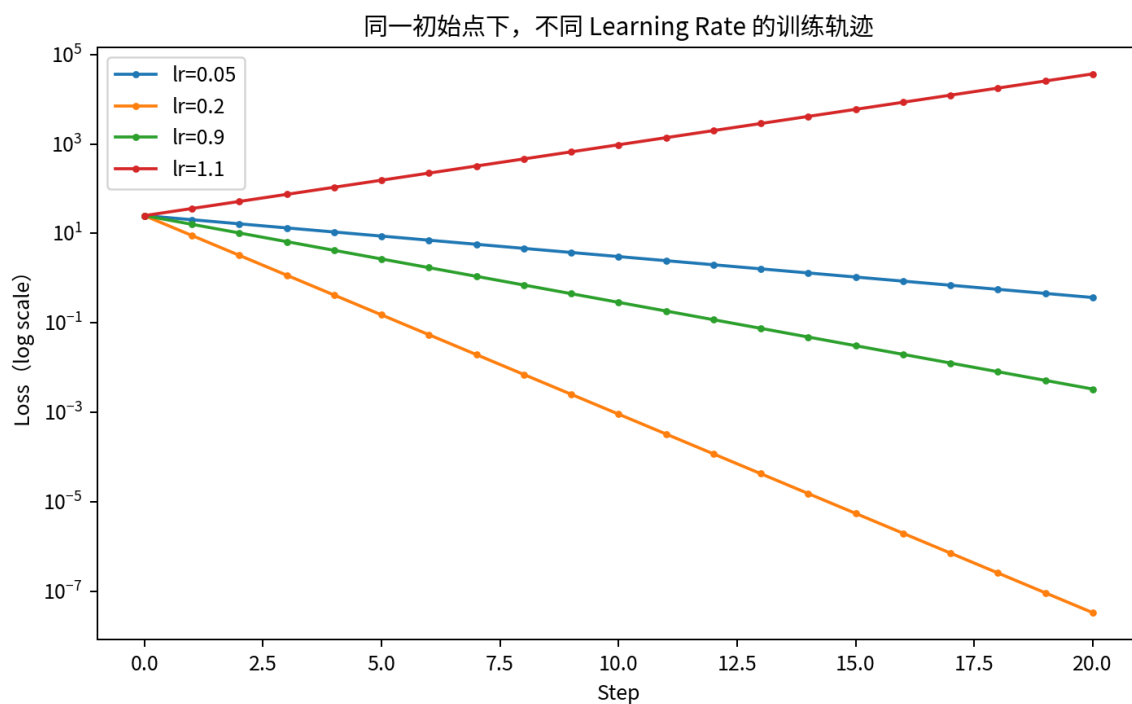


图 3: 同一初始点、同一 Loss 下，不同 Learning Rate 的轨迹

Learning Rate 太大时，“负梯度方向”本身可能没有错，错的是一次走出了局部近似有效的区域。

10 为什么 Gradient 只是一阶局部信息？

第 20 讲讲过：

$$\Delta L \approx \nabla L^T \Delta \theta$$

这是当前点附近的一阶近似。

实验在 $w_0 = 8$ 处，Gradient=10：

Δw	一阶预测 ΔL	实际 ΔL	差异
-0.01	-0.1000	-0.0999	0.0001
-0.10	-1.0000	-0.9900	0.0100
-0.50	-5.0000	-4.7500	0.2500
-2.00	-20.0000	-16.0000	4.0000
-5.00	-50.0000	-25.0000	25.0000

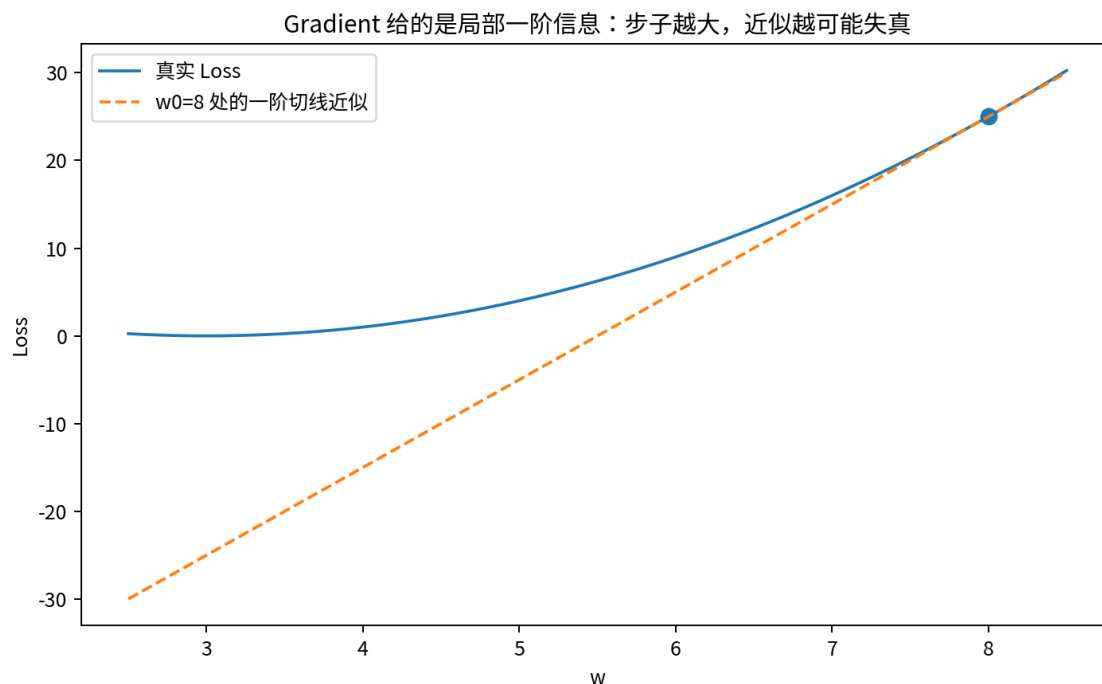


图 4: Gradient 对 Loss 的描述本质上是当前点附近的一阶切线信息

小步时切线近似很准；大步时真实曲线已经明显弯曲，切线预测就会偏离。

11 多个 Parameter 的一步更新：Weight 与 Bias 同时怎样变？

考虑最简单线性模型：

$$\hat{y} = wx + b$$

当前：

$$w = 1, \quad b = 0, \quad x = 2, \quad y = 5$$

则：

$$\hat{y} = 2$$

使用平方误差：

$$L = (\hat{y} - y)^2 = (2 - 5)^2 = 9$$

先求：

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = -6$$

然后：

$$\frac{\partial \hat{y}}{\partial w} = x = 2$$

所以：

$$\frac{\partial L}{\partial w} = -6 \times 2 = -12$$

Bias：

$$\frac{\partial \hat{y}}{\partial b} = 1$$

所以：

$$\frac{\partial L}{\partial b} = -6$$

如果：

$$\eta = 0.1$$

则：

$$w_{new} = 1 - 0.1(-12) = 2.2$$

$$b_{new} = 0 - 0.1(-6) = 0.6$$

注意： w 和 b 是两个不同 Parameter，它们共享同一个 Learning Rate，但各自乘自己的 Gradient。

推广到神经网络矩阵

例如：

$$W_1.shape = (768, 3072), \quad b_1.shape = (3072,)$$

Backward 后：

$$W_1.grad.shape = (768, 3072)$$

$$b_1.grad.shape = (3072,)$$

最基础的 Gradient Descent 就是逐元素：

$$W_1 \leftarrow W_1 - \eta W_1.grad$$

$$b_1 \leftarrow b_1 - \eta b_1.grad$$

即使参数有几百万、几百亿个，底层“每个 Parameter 有对应 Gradient，再生成更新量”的逻辑没有改变。

12 从一个参数推广到很多参数

真实神经网络不是一个 w ，而是大量参数：

$$\theta = [\theta_1, \theta_2, \dots, \theta_n]$$

Gradient：

$$\nabla L = \left[\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_n} \right]$$

更新就是逐元素：

$$\theta_i \leftarrow \theta_i - \eta \frac{\partial L}{\partial \theta_i}$$

如果参数本身是矩阵 W ：

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

这里：

$$W.shape = \left(\frac{\partial L}{\partial W} \right).shape$$

因此可以逐元素更新。

13 【二维图解】 向量参数怎样一起走？

考虑：

$$L(w_1, w_2) = (w_1 - 2)^2 + 4(w_2 + 1)^2$$

Gradient：

$$\nabla L = [2(w_1 - 2), 8(w_2 + 1)]$$

从：

$$(-3, 3)$$

出发，用 $\eta = 0.12$ 更新 30 步，实验最后：

$$(w_1, w_2) \approx (1.99867, -1)$$

$$L \approx 1.77 \times 10^{-6}$$

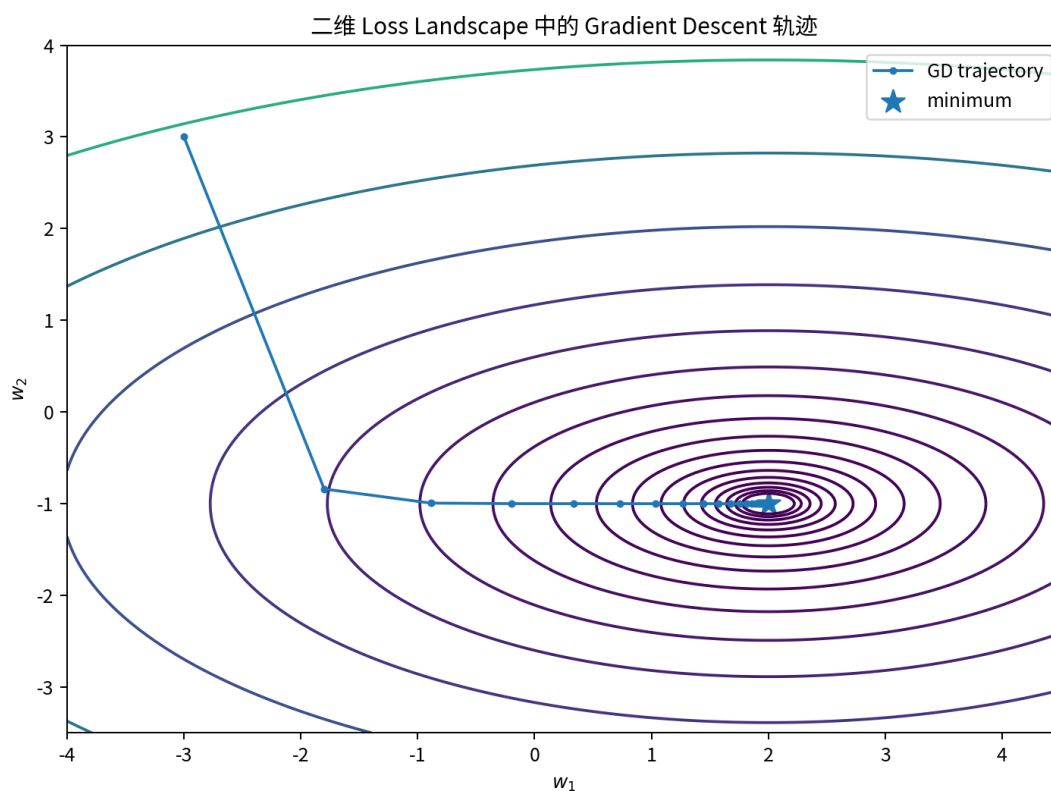


图 5: 二维 Loss Landscape 中的 Gradient Descent 轨迹

这里两个参数并不是轮流更新，而是在每个 Step 根据同一个当前点的 Gradient 一起更新。

14 Gradient Descent 与 PyTorch `optimizer.step()` 的关系

如果最简单地写：

```
p = torch.tensor(8.0, requires_grad=True)
optimizer = torch.optim.SGD([p], lr=0.1)
```

```
loss = (p - 3.0) ** 2

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

那么 backward 后：

$$p.grad = 10$$

而最普通的 SGD step 就会做：

$$p \leftarrow p - 0.1 \times 10 = 7$$

本讲实验验证：手工更新与 torch.optim.SGD 的结果差异为 0。

因此，Optimizer 不是“替代了数学”。它只是把更新规则对模型里所有 Parameter 自动执行。

15 Full-Batch Gradient Descent 是什么？

本讲为了建立纯净基线，最后用所有 41 个样本一起算一次 Loss：

$$X_{all} \rightarrow model \rightarrow L_{all}$$

然后：

$$L_{all}.backward()$$

再更新一次参数。

这意味着：

$$1 \text{ Step} = \text{Dataset} \rightarrow \text{Gradient}$$

实验用线性模型学习：

$$y = 2x + 1$$

最终学到：

$$weight \approx 2, \quad bias \approx 1$$

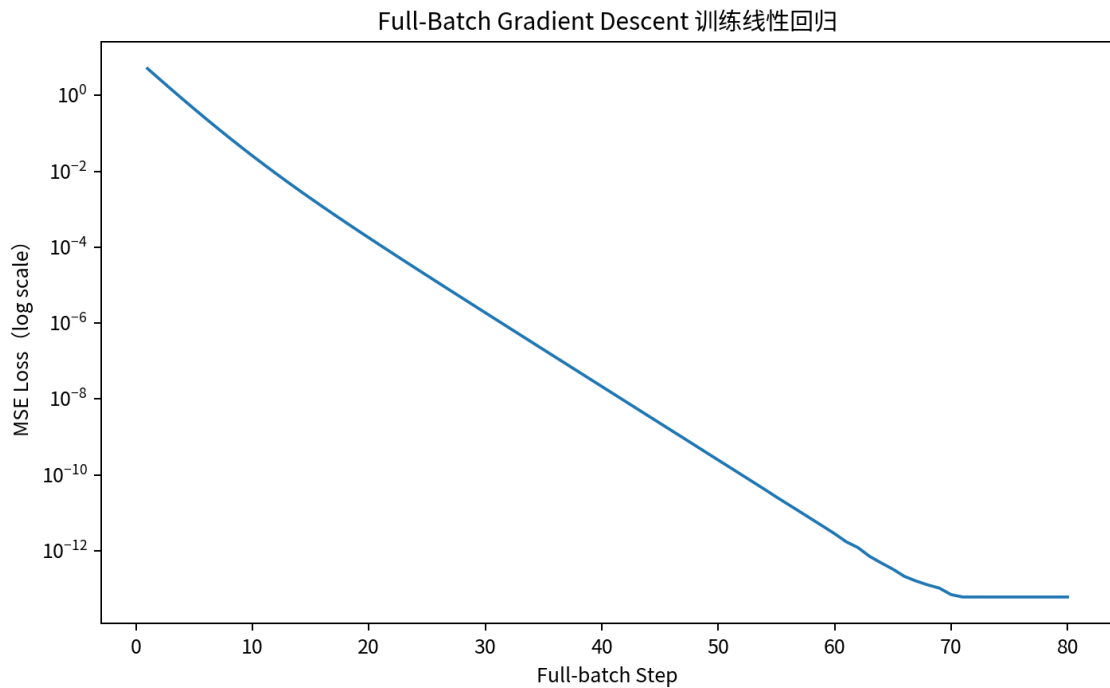


图 6: Full-Batch Gradient Descent 训练线性回归

下一讲的 SGD / Mini-Batch SGD 会改变“每个 Step 用哪些数据来估计 Gradient”。不要把 Gradient Descent 这个更新思想和“每步使用全数据还是一部分数据”混为同一问题。

16 Gradient Descent、SGD、Mini-Batch SGD 到底是什么关系？

从术语上区分：

- Gradient Descent：泛指沿负梯度方向更新参数的基本思想；
- Full-Batch GD：每一步用全部训练数据计算 Gradient；
- Stochastic Gradient Descent：经典定义中，每一步只用随机单个 Sample；
- Mini-Batch SGD：每一步用随机小批 Sample，现代深度学习最常见。

现实中 PyTorch 的类名叫：

```
torch.optim.SGD
```

但你究竟是 Full-Batch、single-sample 还是 Mini-Batch，主要取决于每次 Forward/Backward 喂给它的数据。

17 把公式逐行翻译成 PyTorch 训练代码

最小训练代码：

```
optimizer.zero_grad()
pred = model(x)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
```

现在逐行翻译成数学：

1. **optimizer.zero_grad()** 清掉上一个 Step 留在：

$$\theta.grad$$

里的 Gradient。它没有计算 Forward，也没有更新参数。

2. **pred = model(x)** 使用当前：

$$\theta_t$$

做 Forward，得到 Prediction。

3. **loss = loss_fn(pred,y)** 得到当前：

$$L(\theta_t)$$

4. **loss.backward()** 计算：

$$\nabla L(\theta_t)$$

并把各 Parameter 的梯度写入：

```
parameter.grad
```

到这里 Parameter 本身还没有变化。

5. **optimizer.step()** 如果 Optimizer 是最简单的 SGD，近似就是对所有 Parameter 执行：

$$\theta \leftarrow \theta - \eta \theta.grad$$

这一步完成后才有新的：

$$\theta_{t+1}$$

下一轮 Forward 使用的是新参数。

因此调试训练代码时，应分别检查：Loss 是否合理、Gradient 是否存在/是否异常、Parameter 在 step 前后是否真的改变。只看最终 Loss 往往无法定位是哪一环出了问题。

18 怎样从训练曲线初步判断 Gradient Descent 出了什么问题？

下面只是第一层诊断，不是绝对结论：

现象	第一批应该检查的原因
Loss 稳定下降但很慢	Learning Rate 太小、Gradient 很小、模型容量不足等
Loss 大幅上下震荡	Learning Rate 偏大、Mini-Batch Noise、数据尺度问题等
Loss 很快变成巨大值 / NaN	Learning Rate 过大、Gradient Explosion、数值不稳定等
Loss 长时间几乎不变	Gradient 接近 0、参数未更新、数据/标签问题、代码断图等
训练 Loss 降、验证 Loss 升	更像 Overfitting / Generalization 问题，而非单纯“GD 不工作”

科研记录不要只保存最后一个指标。至少保留 Training/Validation Loss 曲线、Learning Rate、Global Step、随机种子和关键训练设置。否则训练失败时很难判断发生在哪个阶段。

19 【与 LLM / Transformer 的关系】

大模型训练表面上复杂很多，但最底层仍然存在：

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

其中 $\Delta\theta_t$ 最终仍由 Gradient 和 Optimizer 规则决定。

例如 LLM 预训练：

Token Batch → *Transformer* → *Logits* → *Cross Entropy* → *Backward* → *Gradient* → *Optimizer* → *Parameter*

今天学的纯 Gradient Descent 是这一条链里最简单的更新规则。后面学习 Momentum、Adam、AdamW，只是在“有了 Gradient 后怎样生成更新量”这一部分加入更复杂的状态和缩放。

20 【与 Agent Systems 科研的关系】

Agent Systems 论文不一定训练基础模型，但你仍会遇到：

- 微调一个 policy / planner / reward model；
- LoRA / QLoRA 更新；
- 训练 routing / selector / verifier；
- 比较不同 optimization setting；
- 阅读别人训练失败或不稳定的原因。

这时必须能区分：问题来自数据、Loss、Gradient、Learning Rate、Optimizer，还是模型结构，而不是笼统地说“训练没收敛”。

科研实验中比较两个优化方法时，应尽量控制 Initialization、Data split、Batch order、训练预算、Scheduler 等变量。否则“Optimizer A 更好”可能只是起点或数据顺序不同。

21 【科研判断】Training Loss 下降到底能说明什么？

Training Loss 下降能支持：

- 当前代码至少能够把训练目标优化下去；
- Gradient、参数更新、Learning Rate 在当前实验中没有完全失效；
- 模型对训练数据的拟合能力在提高。

但它不能单独证明：

- Validation / Test 表现变好；
- Generalization 好；
- 模型真正学到了目标机制；
- 没有 Data Leakage；
- 学到的表示可迁移；
- Agent 在真实环境中更可靠。

22 【常见误区】

1. **误区：Gradient Descent 就是 Backpropagation。** Backprop 负责算 Gradient, GD 负责用 Gradient 更新。
2. **误区：负梯度方向一定让 Loss 降。** 只在足够小的局部步长意义下可靠，Learning Rate 过大可发散。
3. **误区：Learning Rate 越大越快。** 太大会振荡、越过谷底甚至爆炸。
4. **误区：Learning Rate 越小越安全。** 太小可能在有限训练预算内几乎学不动。
5. **误区：Gradient 小就是训练完成。** 第 21 讲已看到 plateau / saddle point 等反例。
6. **误区：训练 Loss 下降就是模型好。** 训练优化和泛化是不同问题。
7. **误区：torch.optim.SGD 决定了你一定在做 Mini-Batch SGD。** 数据批次决定每步 Gradient 用多少 Sample 估计。

23 【代码实验】本讲脚本在验证什么？

完整实验包含：

1. 标量 Loss 的一步手工 Gradient Descent；
2. 连续 25 步更新，看参数和 Loss 如何变化；
3. 固定同一初始化比较 4 种 Learning Rate；

4. 二维参数空间中的 Gradient Descent 轨迹；
5. 手工更新与 `torch.optim.SGD` 一步严格对照；
6. Full-Batch Gradient Descent 训练 $y = 2x + 1$ ；
7. 通过切线近似观察为什么“大步更新”可能不再符合局部 Gradient 信息。

24 本讲最低闭环能力

如果只保留最核心内容，请确保你能回答：

1. 给出 w 、Gradient 和 Learning Rate，你能手算更新后的 w ；
2. 你能解释公式为什么有负号；
3. 你能解释为什么 Learning Rate 太大会失败；
4. 你能区分 backward 与 step；
5. 你能解释 Full-Batch GD 与下一讲 Mini-Batch SGD 的数据差别；
6. 你知道 Training Loss 下降不能直接推出 Generalization 好。

本讲小结

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

是本讲唯一真正需要长期记住的公式，但必须连同它的含义一起记住：

当前参数 → 算 Gradient → 取负方向 → 乘 Learning Rate → 更新参数 → 重新计算

下一讲：第 34 讲《SGD 与 Mini-Batch SGD》。我们将不再改变“参数沿梯度更新”这一核心，而是改变“每一步 Gradient 到底由哪些样本估计”，从而理解随机性、Gradient Noise 和 Mini-Batch 为什么成为现代深度学习训练的默认方式。